AI ASSISTED CODING

Assignment-19.1

Task 1 – Python to Java Conversion

Task:

Translate a simple Python class into Java using AI assistance.

Instructions:

• Prompt AI to convert a Python class with attributes and methods into equivalent Java code.

• Verify that constructors, method signatures, and data types are properly handled.

• Run both versions (Python & Java) to compare expected outputs.

Expected Output:

• Equivalent working Java class translated from Python.

ommands + Code + Text Run all

Task 1: Python to Java Conversion

First, here's a sample Python class:

[15]
0s

```python
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def get_name(self):
        return self.name

    def get_age(self):
        return self.age

    def celebrate_birthday(self):
        self.age += 1
        print(f"{self.name} is now {self.age} years old!")

    def describe(self):
        return f"Name: {self.name}, Age: {self.age}"


# Example usage of the Python class
print("--- Python Class Example ---")
person1 = Person("Alice", 30)
print(person1.describe())
person1.celebrate_birthday()
print(person1.describe())

person2 = Person("Bob", 25)
print(person2.describe())
```

```
--- Python Class Example ---
Name: Alice, Age: 30
Alice is now 31 years old!
Name: Alice, Age: 31
Name: Bob, Age: 25
```

AI-Generated Java Equivalent

Here's the equivalent Java code for the `Person` class. Key differences include explicit type declarations, `public` access modifiers, a constructor with the class name, `this` keyword for instance variables, and standard `getter` and `setter` patterns.

To run this Java code:

1. Save the code below as `Person.java` in a file on your local machine.
2. Open a terminal or command prompt.
3. Navigate to the directory where you saved `Person.java`.
4. Compile the Java code: `javac Person.java`
5. Run the compiled code: `java Person`

```
[22]
%%bash
cat <<EOF > Person.java
public class Person {
    String name;
    int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String describe() {
        return "Name: " + name + ", Age: " + age;
    }

    public static void main(String[] args) {
        Person p = new Person("Alice", 30);
        System.out.println(p.describe());
    }
}
EOF

javac Person.java
java Person
```

```
javac Person.java
java Person

Name: Alice, Age: 30
```

Task 2 – C++ to Python Function Conversion

Task:

Convert a C++ function for factorial calculation into Python.

Instructions:

• Ask AI to translate a recursive C++ factorial function into

Python.

• Validate correctness by testing with sample inputs.

• Ensure Python version follows Pythonic conventions.

Expected Output:

• A correct Python implementation equivalent to the original

C++ factorial function.

[23]
1s

```
%%bash
cat <<EOF > factorial.cpp
#include <iostream>

// Recursive function to calculate factorial
long long factorial(int n) {
    if (n == 0 || n == 1) {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}

int main() {
    int num = 5;
    std::cout << "Factorial of " << num << " is " << factorial(num) << std::endl;

    num = 10;
    std::cout << "Factorial of " << num << " is " << factorial(num) << std::endl;
    return 0;
}
EOF

g++ factorial.cpp -o factorial
./factorial
```

```
Factorial of 5 is 120
Factorial of 10 is 3628800
```

AI-Generated Python Equivalent

Here's the Python equivalent of the recursive factorial function. Key differences include:

- No explicit type declarations for parameters or return values.
- Function definition uses `def` instead of a return type.
- Conditional statements use `if` / `elif` / `else` with colons and indentation.
- The `main` function concept is replaced by direct function calls or a `if __name__ == '__main__':` block.

```python
def factorial(n):
    """Calculates the factorial of a non-negative integer recursively."""
    if n < 0:
        raise ValueError("Factorial is not defined for negative numbers.")
    elif n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)


# Validate correctness with sample inputs
print("--- Python Function Example ---")

num1 = 5
result1 = factorial(num1)
print(f"Factorial of {num1} is {result1}")

num2 = 10
result2 = factorial(num2)
print(f"Factorial of {num2} is {result2}")

num3 = 0
result3 = factorial(num3)
print(f"Factorial of {num3} is {result3}")
```

```python
print(f"Factorial of {num1} is {result1}")

num2 = 10
result2 = factorial(num2)
print(f"Factorial of {num2} is {result2}")

num3 = 0
result3 = factorial(num3)
print(f"Factorial of {num3} is {result3}")

try:
    factorial(-1)
except ValueError as e:
    print(f"Error for negative input: {e}")
```

```
--- Python Function Example ---
Factorial of 5 is 120
Factorial of 10 is 3628800
Factorial of 0 is 1
Error for negative input: Factorial is not defined for negative numbers.
```

Task 3 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in

Python.

Instructions:

• Provide AI with a SQL query (e.g., SELECT name, salary

FROM employees WHERE salary > 50000).

• Ask AI to generate equivalent Pandas code.

• Test the generated code on a sample DataFrame.

Expected Output:

• Equivalent Pandas query that retrieves the correct subset of

data.

Here's a sample SQL query we'll translate:

```sql
SELECT name, salary
FROM employees
WHERE salary > 50000
ORDER BY salary DESC;
```

First, let's create a sample Pandas DataFrame that resembles an `employees` table.

[25]
0s

```python
import pandas as pd

# Create a sample DataFrame
data = {
    'id': [1, 2, 3, 4, 5, 6],
    'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank'],
    'age': [30, 24, 35, 40, 28, 32],
    'salary': [60000, 45000, 75000, 52000, 48000, 65000],
    'department': ['HR', 'Engineering', 'Sales', 'HR', 'Engineering', 'Sales']
}
employees_df = pd.DataFrame(data)

print("Original DataFrame:")
display(employees_df)
```

```
Original DataFrame:
```

	id	name	age	salary	department
0	1	Alice	30	60000	HR
1	2	Bob	24	45000	Engineering
2	3	Charlie	35	75000	Sales
3	4	David	40	52000	HR
4	5	Eve	28	48000	Engineering
5	6	Frank	32	65000	Sales

Next steps: Generate code with `employees_df` New interactive sheet

```python
# Equivalent Pandas code for:
# SELECT name, salary FROM employees WHERE salary > 50000 ORDER BY salary DESC;

# 1. Filter rows where salary > 50000
filtered_employees = employees_df[employees_df['salary'] > 50000]

# 2. Select 'name' and 'salary' columns
selected_columns = filtered_employees[['name', 'salary']]

# 3. Order by salary in descending order
pandas_result = selected_columns.sort_values(by='salary', ascending=False)

print("\nResulting Pandas DataFrame (equivalent to SQL query):")
display(pandas_result)
```

```
Resulting Pandas DataFrame (equivalent to SQL query):
```

	name	salary
2	Charlie	75000
5	Frank	65000
0	Alice	60000
3	David	52000

Next steps: Generate code with pandas_result | New interactive sheet

Task 4: Multi-Language Snippet Converter - Python to JavaScript: Reverse String

We will translate a simple Python function that reverses a string into its JavaScript equivalent.

[27]
0s

```python
# Python function to reverse a string
def reverse_string_python(s):
    return s[::-1]

# Test the Python implementation
python_test_string = "Hello, Colab!"
reversed_python_string = reverse_string_python(python_test_string)
print(f"Original Python String: '{python_test_string}'")
print(f"Reversed Python String: '{reversed_python_string}'")
```

```
Original Python String: 'Hello, Colab!'
Reversed Python String: '!baloC ,olleH'
```

[29]
0s

```javascript
%%javascript
// JavaScript function to reverse a string
function reverseStringJavaScript(s) {
    return s.split('').reverse().join('');
}

// Test the JavaScript implementation
const jsTestString = "Hello, Colab!";
const reversedJsString = reverseStringJavaScript(jsTestString);
console.log(`Original JavaScript String: '${jsTestString}'`);
console.log(`Reversed JavaScript String: '${reversedJsString}'`);
```

...